

Departamento Electrónica — Técnicas Digitales II

Actividad de Formación Práctica N°4 — LAB

Tema: Retardos no bloqueantes.

Profesor: Ing. Rubén Darío Mansilla **ATTP:** Ing. Lucas Abdala

AFP 4 — Grupo 3 — TDII 2026

Integrantes:

- Plaate Iván
 - *(completar compañeros)*
 - *(completar compañeros)*
-

1. Introducción

El presente informe documenta el desarrollo de la AFP N°4 de laboratorio, cuyo objetivo fue implementar un **driver de retardos no bloqueantes** (`API_delay`) para la placa de desarrollo **STM32 NUCLEO-F429ZI**, e integrarlo en las aplicaciones desarrolladas en la práctica anterior, reemplazando la función bloqueante `HAL_Delay()`.

La práctica sigue el esquema acumulativo de la materia: el driver `API_GPIO` creado en la AFP3 se cargó en los nuevos proyectos **sin ninguna modificación**, y `API_delay` se sumó como una nueva capa de servicio. El objetivo de diseño fue nuevamente la **portabilidad total**: `API_delay` no depende de ningún pin, puerto ni periférico concreto, por lo que puede reutilizarse en cualquier aplicación.

2. Desarrollo del driver `API_delay`

El driver se creó respetando la estructura de la cátedra:

```
Drivers/  
└─ API/  
    ├── Inc/  
    │   ├── API_GPIO.h  
    │   └─ API_delay.h  
    └─ Src/  
        ├── API_GPIO.c  
        └─ API_delay.c
```

2.1. Tipos de datos exportados

```

typedef uint32_t tick_t;      /* Tipo base de tiempo, en milisegundos */
typedef bool bool_t;         /* Tipo booleano */

typedef struct
{
    tick_t startTime; /* Instante de inicio del retardo (HAL_GetTick) */
    tick_t duration;  /* Duracion configurada [ms] */
    bool_t running;   /* true = retardo en curso */
} delay_t;

```

Cada `delay_t` es una instancia de retardo completamente **independiente**: la aplicación puede declarar tantos retardos simultáneos como necesite, algo imposible de lograr con `HAL_Delay()`.

2.2. Funciones exportadas

Función	Descripción
<code>delayInit(delay_t *d, tick_t duration)</code>	Inicializa la estructura con la duración indicada; el retardo queda detenido (<code>running = false</code>)
<code>delayRead(delay_t *d)</code>	Si el retardo está detenido, lo arma y devuelve <code>false</code> . Si está en curso, evalúa si transcurrió la duración: si se cumplió devuelve <code>true</code> y lo detiene; si no, devuelve <code>false</code>
<code>delayWrite(delay_t *d, tick_t duration)</code>	Permite modificar la duración de un retardo existente sin reinicializarlo

La base temporal es `HAL_GetTick()` (tick de 1 ms del SysTick configurado por HAL). El uso típico es consultar `delayRead()` dentro del lazo principal: cuando el retardo se cumple devuelve `true` una vez, y para reiniciarlo basta volver a llamar a la función (auto-rearme en el próximo ciclo).

2.3. Diferencia con el retardo bloqueante

`HAL_Delay(t)` detiene la ejecución del programa durante `t` milisegundos: el CPU queda "congelado" esperando y no puede atender ninguna otra tarea. En cambio, `delayRead()` **nunca detiene el lazo principal**: solo informa si el tiempo transcurrió. Esto permite que el `while(1)` siga ejecutando el resto de la lógica — lectura del pulsador, otras tareas, otras secuencias — mientras el retardo transcurre en paralelo.

3. Aplicaciones modificadas

Se refactorizaron las cuatro aplicaciones de la AFP3 con retardos no bloqueantes, respetando el formato de nombre `App_4_Y_Grupo_3_2026`. En todas ellas se eliminó por completo el uso de `HAL_Delay()`: el lazo principal corre de manera continua y el pulsador se lee en **cada iteración**, por lo que la respuesta al botón es inmediata (ya no depende de pasos de lectura dentro de la espera).

3.1. App_4_1 — Secuencia básica no bloqueante

Enciende y apaga secuencialmente los tres LEDs onboard (LD1 → LD2 → LD3), 200 ms encendido y 200 ms apagado por LED, en forma circular. La alternancia se produce cuando `delayRead()` informa el cumplimiento del retardo, sin detener el lazo.

3.2. App_4_2 — Inversión de sentido

Misma secuencia que App_4_1, pero el pulsador de usuario invierte el sentido de recorrido cada vez que se presiona. Al leerse el pulsador en cada iteración del lazo, la inversión se percibe de inmediato, incluso en medio de un tiempo de espera.

3.3. App_4_3 — Cuatro secuencias alternadas

El pulsador alterna circularmente entre cuatro secuencias:

- **Secuencia 1:** encendido secuencial con alternancia de 150 ms.
- **Secuencia 2:** los tres LEDs parpadean simultáneamente con alternancia de 300 ms.
- **Secuencia 3:** parpadeo con períodos independientes — LD1 a 100 ms, LD2 a 300 ms y LD3 a 600 ms. Se implementó con **tres instancias de `delay_t` simultáneas**, una por LED, aprovechando que cada retardo es independiente. En la versión anterior (AFP3) esta secuencia requería un tick base con contadores; con `API_delay` cada LED tiene su propio retardo y la lógica quedó directa.
- **Secuencia 4:** LD1 y LD3 parpadean juntos mientras LD2 lo hace en forma inversa, con alternancia de 150 ms.

Al cambiar de secuencia se apagan todos los LEDs, se reinician las variables de estado y se reinician los retardos, de modo que cada secuencia arranca desde una condición conocida.

3.4. App_4_4 — Frecuencia de parpadeo variable

Los tres LEDs parpadean simultáneamente y el pulsador cambia el tiempo de alternancia de forma secuencial entre cuatro valores predefinidos: 100 ms → 250 ms → 500 ms → 1000 ms → vuelta a 100 ms. El cambio se aplica con `delayWrite()` sobre el retardo en curso, sin necesidad de reinicializar la estructura.

4. Observaciones

- Los drivers se **acumulan**: cada proyecto carga `API_GPIO` (sin modificar desde AFP3) más `API_delay`. Esto materializa la filosofía de la materia de cargar el driver ya pulido en la práctica siguiente.
- `API_delay` es completamente portable: solo depende de `HAL_GetTick()`, por lo que sirve para cualquier retardo no bloqueante en cualquier placa con HAL.
- Se eliminaron todas las llamadas `HAL_Delay()` de la lógica de aplicación; el programa nunca queda detenido.
- La App_4_3 es el ejemplo más claro de la ventaja del no bloqueante: tres retardos independientes corriendo en simultáneo, algo que el enfoque bloqueante no permite.

- El rebote mecánico del pulsador aún no se filtra en esta práctica; el antirrebote por máquina de estados corresponde a la AFP5.

5. Repositorio

Repositorio grupal en GitHub:

https://github.com/ivanplaate-oss/Grupo_3_TDII_2026

Las aplicaciones se encuentran en la carpeta `AFP_4_TDII_2026/` :

- `AFP_4_TDII_2026/App_4_1_Grupo_3_2026` — secuencia básica no bloqueante
- `AFP_4_TDII_2026/App_4_2_Grupo_3_2026` — inversión de sentido
- `AFP_4_TDII_2026/App_4_3_Grupo_3_2026` — cuatro secuencias alternadas
- `AFP_4_TDII_2026/App_4_4_Grupo_3_2026` — frecuencia de parpadeo variable

6. Conclusiones

Se implementó un driver de retardos no bloqueantes portable y se lo integró en las cuatro aplicaciones junto con el driver `API_GPIO` ya desarrollado, verificando el funcionamiento correcto en la placa NUCLEO-F429ZI. El reemplazo de `HAL_Delay()` por retardos consultables liberó el lazo principal, mejoró la respuesta al pulsador y habilitó retardos independientes en paralelo. El driver quedó listo para ser reutilizado por el driver de antirrebote de la AFP5, que lo emplea internamente para validar los flancos del pulsador.